🚀 Complete Blog Backend API Documentation

A comprehensive guide to all RESTful endpoints available in the Blog Backend API.

📌 Base Information

- **Base URL:** `http://localhost:5000/api/v1`
- **Content-Type:** `application/json` (unless specified otherwise)
- **Authentication:** JWT via `Authorization: Bearer <token>`
- **Rate Limiting:** General endpoints (100 req/15min), Auth endpoints (20 req/15min)

🔐 1. Authentication & User Profile APIs

1.1 User Registration

Registers a new user account and triggers an OTP verification email.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/register`
- **Auth Required:** ❌ No

Request Body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "SecurePass123",
  "avatar": "[https://example.com/avatar.jpg](https://example.com/avatar.jpg)",
  "bio": "I love blogging!"
}
```

1.2 Verify Email OTP

Verifies the user's email using the OTP sent during registration.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/verify-otp`
- **Auth Required:** ❌ No

Request Body:

```
{
  "email": "john@example.com",
  "otp": "12345"
}
```

1.3 Resend OTP

Resends the verification OTP to the user's email.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/resend-otp`
- **Auth Required:** ❌ No

Request Body:

```
{
  "email": "john@example.com"
}
```

1.4 User Login

Authenticates a user and returns a JWT token and user profile.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/login`
- **Auth Required:** ❌ No

Request Body:

```
{
  "email": "john@example.com",
  "password": "SecurePass123"
}
```

1.5 User Logout

Invalidates the user's current session/token.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/logout`
- **Auth Required:** ✅ Yes (`Authorization: Bearer <token>`)

1.6 Get Current User Profile

Retrieves the profile data of the currently authenticated user.

- **Endpoint:** `GET http://localhost:5000/api/v1/auth/me/profile`
- **Auth Required:** ✅ Yes

1.7 Get User by ID (Public Profile)

Retrieves the public profile information of a specific user.

- **Endpoint:** `GET http://localhost:5000/api/v1/auth/:id`
- **Auth Required:** ❌ No
- **Path Parameter:** `id` (string, required) - MongoDB User ID.

1.8 Update Profile

Updates the authenticated user's profile details.

- **Endpoint:** `PUT http://localhost:5000/api/v1/auth/me/update-profile`
- **Auth Required:** ✅ Yes

Request Body:

```
{
  "name": "John Doe Updated",
  "bio": "Updated bio text here",
  "socialLinks": {
    "twitter": "[https://twitter.com/johndoe](https://twitter.com/johndoe)"
  },
  "preferences": {
    "emailNotifications": true
  }
}
```

1.9 Upload Profile Image

Uploads a new avatar for the authenticated user.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/me/upload-profile-image`
- **Auth Required:** ✅ Yes
- **Content-Type:** `multipart/form-data`
- **Form Data:** `image` (file, required)

1.10 Change Password

Changes the authenticated user's password.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/me/change-password`
- **Auth Required:** ✅ Yes

Request Body:

```
{
  "currentPassword": "OldPass123",
  "newPassword": "NewSecurePass123",
  "confirmPassword": "NewSecurePass123"
}
```

1.11 Forgot Password

Initiates the password reset process by sending a reset link/token.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/forgot-password`
- **Auth Required:** ❌ No

Request Body:

```
{
  "email": "john@example.com"
}
```

1.12 Reset Password

Resets the password using the token provided via email.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/reset-password/:token`
- **Auth Required:** ❌ No
- **Path Parameter:** `token` (string, required) - Reset token.

Request Body:

```
{
  "password": "NewSecurePass123",
  "confirmPassword": "NewSecurePass123"
}
```

1.13 Delete Account

Permanently deletes the authenticated user's account.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/me/delete-account`
- **Auth Required:** ✅ Yes

Request Body:

```
{
  "password": "YourPassword123"
}
```

1.14 Follow/Unfollow User

Toggles the follow status for a specific user.

- **Endpoint:** `POST http://localhost:5000/api/v1/auth/me/follow/:userId`
- **Auth Required:** ✅ Yes
- **Path Parameter:** `userId` (string, required) - ID of the user to follow/unfollow.

📝 2. Post Management APIs

2.1 Create Post

Creates a new blog post.

- **Endpoint:** `POST http://localhost:5000/api/v1/posts/create`

- **Auth Required:** ✅ Yes

Request Body:

```
{
  "title": "Getting Started with Node.js",
  "content": "Node.js is a powerful JavaScript runtime...",
  "description": "Learn the basics of Node.js development",
  "tags": ["nodejs", "backend", "javascript"],
  "category": "Technology"
}
```

2.2 Get All Posts

Retrieves a paginated list of all published posts with optional filtering.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts`
- **Auth Required:** ❌ No
- **Query Params:** `page`, `limit`, `tag`, `search`, `author`, `category`, `sort`, `isFeatured`
- **Example:** `GET http://localhost:5000/api/v1/posts?page=1&limit=10&sort=popular`

2.3 Get Top Posts

Retrieves the most popular posts based on views/engagement.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/top`
- **Auth Required:** ❌ No
- **Query Params:** `limit` (default: 5), `timeframe` (week, month, all)

2.4 Get Featured Posts

Retrieves posts manually marked as featured.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/featured`
- **Auth Required:** ❌ No
- **Query Params:** `limit` (default: 10)

2.5 Search Posts

Searches for posts based on a text query.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/search`
- **Auth Required:** ❌ No
- **Query Params:** `q` (required, min 2 chars), `page`, `limit`

2.6 Get Single Post by Slug

Retrieves the full content of a specific post using its URL-friendly slug.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/article/:slug`

- **Auth Required:** ❌ No
- **Path Parameter:** `slug` (string, required)

2.7 Get Posts by Category

Retrieves a paginated list of posts belonging to a specific category.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/category/:category`
- **Auth Required:** ❌ No
- **Path Parameter:** `category` (string, required)
- **Query Params:** `page`, `limit`

2.8 Get Posts by Author

Retrieves all posts written by a specific user.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/author/:authorId`
- **Auth Required:** ❌ No
- **Path Parameter:** `authorId` (string, required)
- **Query Params:** `page`, `limit`

2.9 Get Posts by Tag

Retrieves all posts containing a specific tag.

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/tag/:tag`
- **Auth Required:** ❌ No
- **Path Parameter:** `tag` (string, required)
- **Query Params:** `page`, `limit`

2.10 Get My Posts (Current User)

Retrieves all posts created by the currently authenticated user (including drafts).

- **Endpoint:** `GET http://localhost:5000/api/v1/posts/me/posts`
- **Auth Required:** ✅ Yes
- **Query Params:** `page`, `limit`, `sort`, `status` (draft, published, archived)

2.11 Update Post

Updates an existing post. Only the author or an admin can perform this.

- **Endpoint:** `PUT http://localhost:5000/api/v1/posts/:id`
- **Auth Required:** ✅ Yes
- **Path Parameter:** `id` (string, required) - Post MongoDB ID.

Request Body (Partial updates allowed):

```
{
  "title": "Updated Title",
  "status": "published"
}
```

2.12 Delete Post

Permanently deletes a post.

- **Endpoint:** `DELETE http://localhost:5000/api/v1/posts/:id`
- **Auth Required:** ✅ Yes
- **Path Parameter:** `id` (string, required)

2.13 Like/Unlike Post

Toggles the like status of a post for the authenticated user.

- **Endpoint:** `POST http://localhost:5000/api/v1/posts/:id/like`
- **Auth Required:** ✅ Yes
- **Path Parameter:** `id` (string, required)

2.14 Save/Unsave Post

Bookmarks or removes a bookmark for a post.

- **Endpoint:** `POST http://localhost:5000/api/v1/posts/:id/save`
- **Auth Required:** ✅ Yes
- **Path Parameter:** `id` (string, required)

2.15 Publish Scheduled Posts (Admin)

Admin endpoint to batch publish posts that were scheduled for the future.

- **Endpoint:** `POST http://localhost:5000/api/v1/posts/admin/publish-scheduled`
- **Auth Required:** ✅ Yes (Admin Role Required)

🚦 HTTP Status Codes Guide

Code	Status	Meaning
200	OK	Request was successful.
201	Created	Resource was successfully created.
400	Bad Request	Missing or invalid data provided by the client.
401	Unauthorized	Invalid, expired, or missing JWT token.

403	Forbidden	Valid token, but insufficient permissions (e.g., non-admin).
404	Not Found	The requested resource or endpoint does not exist.
409	Conflict	Data already exists (e.g., email already registered).
410	Gone	Resource is no longer available (e.g., expired OTP).
429	Too Many Requests	Rate limit exceeded. Try again later.
500	Server Error	An unexpected backend error occurred.

💬 Comments API Documentation

Welcome to the official API documentation for the Comments module. This document outlines all available endpoints, required parameters, and expected responses for managing comments and replies.

🔗 Base URL

All endpoints in this section are relative to the following base URL:

```
http://localhost:5000/api/v1/comments
```

🔐 Authentication

Protected and Admin routes require **JSON Web Tokens (JWT)**. Pass the token in the `Authorization` header: `Authorization: Bearer <your_token_here>`

🌍 Public Routes

1. Get All Comments on a Post

- **Method:** `GET`
- **Endpoint:** `/post/:postId`
- **Description:** Retrieve a paginated list of top-level comments for a specific post.
- **Query Parameters:**
  - `page` (default: 1)
  - `limit` (default: 10)
- **Expected Response:**

```
{
  "success": true,
  "total": 25,
  "page": 1,
  "pages": 3,
  "limit": 10,
  "data": [
    {
      "_id": "65f3c2a8b1c2d3e4f5g6h7i8",
      "content": "Great post!",
      "author": {
        "_id": "65f3c2a8b1c2d3e4f5g6h7i7",
        "name": "John Doe",
        "avatar": "https://...",
        "role": "user"
      },
      "post": "65f3c2a8b1c2d3e4f5g6h7i9",
```

```
      "likes": 5,
      "replies": [],
      "isEdited": false,
      "createdAt": "2025-03-21T10:30:00Z",
      "updatedAt": "2025-03-21T10:30:00Z"
    }
  ]
}
```

2. Get Single Comment

- **Method:** `GET`
- **Endpoint:** `/:commentId`
- **Description:** Retrieve detailed information about a single comment.
- **Expected Response:**

```
{
  "success": true,
  "data": {
    "_id": "65f3c2a8b1c2d3e4f5g6h7i8",
    "content": "Great post!",
    "author": {
      "_id": "65f3c2a8b1c2d3e4f5g6h7i7",
      "name": "John Doe",
      "avatar": "https://...",
      "role": "user"
    },
    "post": "65f3c2a8b1c2d3e4f5g6h7i9",
    "likes": 5,
    "likedBy": ["65f3c2a8b1c2d3e4f5g6h7i6"],
    "replies": [],
    "isEdited": false,
    "createdAt": "2025-03-21T10:30:00Z"
  }
}
```

3. Get Comment Replies

- **Method:** `GET`
- **Endpoint:** `/:commentId/replies`
- **Description:** Retrieve a paginated list of replies to a specific parent comment.
- **Query Parameters:**
  - `page` (default: 1)
  - `limit` (default: 5)
- **Expected Response:**

```
{
  "success": true,
  "total": 3,
  "page": 1,
  "pages": 1,
  "data": [
    {
      "_id": "65f3c2a8b1c2d3e4f5g6h7i9",
      "content": "Thanks for your feedback!",
      "author": {
        "_id": "65f3c2a8b1c2d3e4f5g6h7i7",
        "name": "Jane Smith",
        "avatar": "https://...",
        "role": "user"
      },
      "parentComment": "65f3c2a8b1c2d3e4f5g6h7i8",
      "likes": 2,
      "createdAt": "2025-03-21T10:35:00Z"
    }
  ]
}
```

🔒 Protected Routes (Requires Authentication)

4. Create Comment

- **Method:** `POST`
- **Endpoint:** `/post/:postId/create`
- **Description:** Create a new top-level comment on a post.
- **Body:**

```
{
  "content": "This is an amazing article!"
}
```

- **Expected Response:**

```
{
  "success": true,
  "message": "Comment created successfully.",
  "data": {
    "_id": "65f3c2a8b1c2d3e4f5g6h7i8",
    "content": "This is an amazing article!",
    "author": {
      "_id": "65f3c2a8b1c2d3e4f5g6h7i7",
      "name": "John Doe",
      "avatar": "https://...",
      "role": "user"
    },
    "post": "65f3c2a8b1c2d3e4f5g6h7i9",
```

```
    "likes": 0,
    "likedBy": [],
    "createdAt": "2025-03-21T10:40:00Z"
  }
}
```

5. Reply to Comment

- **Method:** `POST`
- **Endpoint:** `/:commentId/reply`
- **Description:** Post a reply to an existing comment.
- **Body:**

```
{
  "content": "I agree with you!"
}
```

- **Expected Response:**

```
{
  "success": true,
  "message": "Reply created successfully.",
  "data": {
    "_id": "65f3c2a8b1c2d3e4f5g6h7i9",
    "content": "I agree with you!",
    "author": {
      "_id": "65f3c2a8b1c2d3e4f5g6h7i6",
      "name": "Jane Smith",
      "avatar": "https://...",
      "role": "user"
    },
    "parentComment": "65f3c2a8b1c2d3e4f5g6h7i8",
    "likes": 0,
    "createdAt": "2025-03-21T10:45:00Z"
  }
}
```

6. Update Comment

- **Method:** `PUT`
- **Endpoint:** `/:commentId`
- **Description:** Edit an existing comment. User must be the author.
- **Body:**

```
{
  "content": "Updated comment text"
```

```
}
```

- **Expected Response:**

```
{
  "success": true,
  "message": "Comment updated successfully.",
  "data": {
    "_id": "65f3c2a8b1c2d3e4f5g6h7i8",
    "content": "Updated comment text",
    "isEdited": true,
    "editedAt": "2025-03-21T10:50:00Z"
  }
}
```

7. Delete Comment (Soft Delete)

- **Method:** `DELETE`
- **Endpoint:** `/:commentId`
- **Description:** Mark a comment as deleted (soft delete). User must be the author.
- **Expected Response:**

```
{
  "success": true,
  "message": "Comment deleted successfully."
}
```

8. Like/Unlike Comment

- **Method:** `POST`
- **Endpoint:** `/:commentId/like`
- **Description:** Toggle the authenticated user's like on a specific comment.
- **Expected Response:**

```
{
  "success": true,
  "message": "Comment liked successfully.",
  "isLiked": true,
  "likes": 6
}
```

🛡️ Admin Routes (Requires Admin Role)

9. Get All Comments (Admin)

- **Method:** `GET`
- **Endpoint:** `/` admin/get-all/comments
- **Description:** Retrieve all comments across the platform with advanced filtering.
- **Query Parameters:**
  - `page` (default: 1)
  - `limit` (default: 20)
  - `postId` (Filter by post ID)
  - `isDeleted` (Filter soft-deleted: "true"/"false")

10. Force Delete Comment

- **Method:** `DELETE`
- **Endpoint:** `/admin/:commentId/force` admin/delete/:commentId/force
- **Description:** Permanently delete a comment from the database.
- **Body:**

```
{
  "reason": "Spam content"
}
```

- **Expected Response:**

```
{
  "success": true,
  "message": "Comment permanently deleted by admin.",
  "data": {
    "deletedCommentId": "65f3c2a8b1c2d3e4f5g6h7i8",
    "reason": "Spam content",
    "deletedBy": "65f3c2a8b1c2d3e4f5g6h7i7",
    "deletedAt": "2025-03-21T10:55:00Z"
  }
}
```

❌ Common Error Responses

Status Code	Description	Example Response
400 Bad Request	Missing or invalid data.	`{"success": false, "message": "Comment content is required."}`
401 Unauthorized	Missing or invalid token.	`{"success": false, "message": "Please login first"}`

403 Forbidden	User lacks permission.	`{"success": false, "message": "You are not authorized to update this comment."}`